

Tratamento de Exceções em Java

Parte 1: Por que tratar erros?

O problema: erros que travam o programa

Em algum momento, todo programa encontra uma situação inesperada: uma divisão por zero, um índice de array que não existe, um dado digitado errado pelo usuário. Sem nenhum tratamento, esses erros **interrompem o programa imediatamente**.

```
1 public class Main {  
2     public static void main(String[] args) {  
3         int a = 10;  
4         int b = 0;  
5  
6         System.out.println("Vou dividir agora...");  
7         int resultado = a / b; // ERRO: divisao por zero!  
8         System.out.println("Resultado: " + resultado); // nunca  
9         executa  
10    }
```

Listing 1: Um erro simples que derruba o programa

Saída no terminal

```
Vou dividir agora...  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at Main.main(Main.java:7)
```

O que aconteceu?

- A linha `a / b` gerou um erro em tempo de execução.
- Como ninguém tratou esse erro, o Java **encerrou o programa abruptamente** e imprimiu uma mensagem técnica (o *stack trace*).
- A linha `System.out.println("Resultado: " + resultado);` **nunca chegou a ser executada**.

Em um programa real, isso significa uma tela travada, um sistema fora do ar, ou dados perdidos — exatamente o que queremos evitar.

O que é uma exceção?

Definição: Exceção

Uma **exceção** é um objeto que representa um evento anormal ocorrido durante a execução do programa. Quando um erro desse tipo acontece, o Java **lança** (*throws*) uma exceção, interrompendo o fluxo normal do código a partir daquele ponto — a menos que exista um trecho de código preparado para **tratá-la**.

Analogia: o alarme de incêndio

Um alarme de incêndio não impede que o fogo comece — mas garante que, quando ele começa, **alguém seja avisado** e um protocolo entre em ação (evacuar, chamar o corpo de bombeiros), em vez de todos continuarem trabalhando normalmente como se nada tivesse acontecido. Uma exceção funciona da mesma forma: ela avisa que algo deu errado e dá ao programa a chance de reagir, em vez de simplesmente travar.

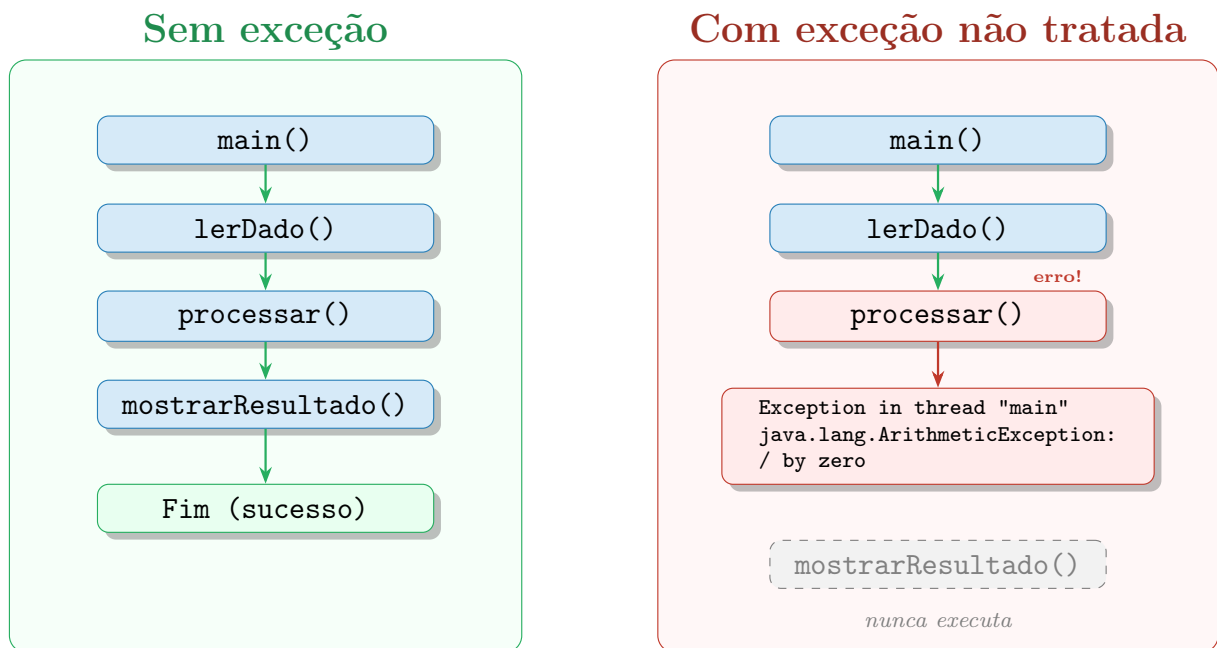


Figure 1: Sem tratamento, uma exceção interrompe o programa imediatamente: o restante do código nunca é executado.

Erros comuns em tempo de execução

Java já possui diversas classes de exceção prontas para representar erros comuns. Conhecer as mais frequentes ajuda a entender rapidamente o que deu errado.

Exceção	Quando ocorre	Exemplo
ArithmeticException	Operação matemática inválida, como divisão inteira por zero.	10 / 0
NullPointerException	Tentativa de usar um objeto que é null.	String s = null; s.length();
ArrayIndexOutOfBoundsException	Acesso a um índice de array que não existe.	int[] v = new int[3]; v[5];
NumberFormatException	Conversão de um texto inválido para número.	Integer.parseInt("abc")

Verifique seu entendimento

Qual dessas exceções seria lançada pelo código `int[] notas = new int[5]; System.out.println(notas[10]);`?

Parte 2: Tratando exceções com try-catch

A estrutura try-catch

Definição: try e catch

O bloco `try` envolve o código que **pode** lançar uma exceção. Se uma exceção ocorrer dentro dele, a execução do `try` é interrompida e o controle passa imediatamente para o bloco `catch` correspondente, que **trata** o erro. Se nenhuma exceção ocorrer, o `catch` é simplesmente ignorado.

```

1 try {
2     // código que pode lançar uma exceção
3 } catch (TipoDaExcecao nomeDaVariavel) {
4     // código executado SOMENTE se a exceção ocorrer
5 }

```

Listing 2: Sintaxe básica de try-catch

```

1 public class Main {
2     public static void main(String[] args) {
3         int a = 10;
4         int b = 0;
5
6         try {
7             int resultado = a / b;
8             System.out.println("Resultado: " + resultado);
9         } catch (ArithmeticException e) {
10            System.out.println("Erro: não é possível dividir por
                zero!");

```

```

11     }
12
13     System.out.println("O programa continua normalmente.");
14 }
15 }

```

Listing 3: Protegendo a divisão com try-catch

Saída esperada

```

Erro: nao e possivel dividir por zero!
O programa continua normalmente.

```

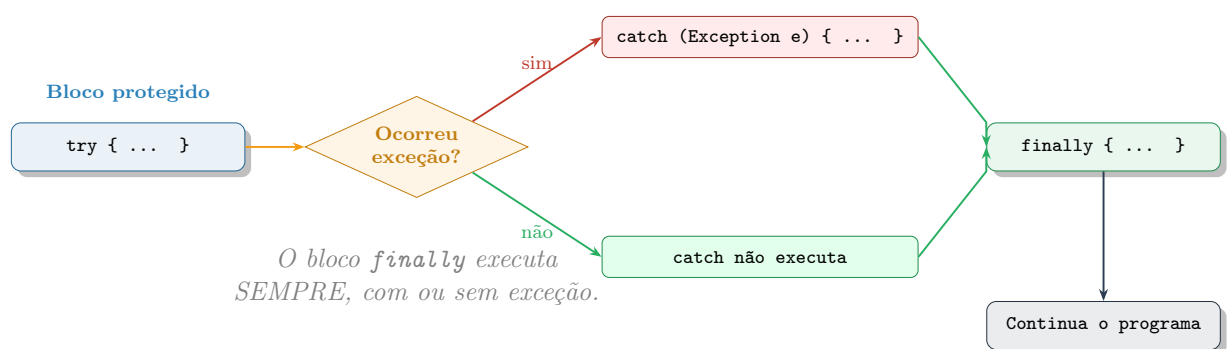


Figure 2: Fluxo de execução de try-catch-finally: o bloco finally roda em qualquer um dos dois caminhos.

Múltiplos catch: tratando erros diferentes de formas diferentes

É possível ter **mais de um bloco catch** para o mesmo try, tratando cada tipo de exceção de um jeito específico.

```

1 public class Main {
2     public static void main(String[] args) {
3         String[] valores = {"10", "abc", "0", "5"};
4
5         for (String valor : valores) {
6             try {
7                 int numero = Integer.parseInt(valor);
8                 int resultado = 100 / numero;
9                 System.out.println("100 / " + numero + " = " +
resultado);
10            } catch (NumberFormatException e) {
11                System.out.println("\"" + valor + "\" nao e um
numero valido.");
12            } catch (ArithmeticException e) {
13                System.out.println("Nao e possivel dividir por zero
.");
14            }
15        }
16    }

```

17 }

Listing 4: Tratando dois tipos de erro na mesma operação

Saída esperada

```
100 / 10 = 10
"abc" nao e um numero valido.
Nao e possivel dividir por zero.
100 / 5 = 20
```

Prefira catch específicos a um catch genérico

É tentador escrever apenas `catch (Exception e)` para capturar “qualquer erro”. Mas isso mistura problemas completamente diferentes no mesmo tratamento e dificulta saber o que realmente aconteceu. Sempre que possível, capture o **tipo mais específico** da exceção esperada.

O bloco finally

Definição: finally

O bloco `finally` é **opcional** e, quando presente, executa **sempre** — tenha ocorrido uma exceção ou não, e mesmo que o `catch` contenha um `return`. É o lugar ideal para código de “limpeza” que precisa rodar de qualquer forma.

```
1 public class Main {
2     public static void main(String[] args) {
3         try {
4             System.out.println("1. Dentro do try");
5             int resultado = 10 / 0;
6             System.out.println("2. Esta linha nao executa");
7         } catch (ArithmeticException e) {
8             System.out.println("3. Dentro do catch: " + e.
9             getMessage());
10        } finally {
11            System.out.println("4. Dentro do finally: executa
12            sempre");
13        }
14        System.out.println("5. Depois do try-catch-finally");
15    }
16 }
```

Listing 5: Ordem de execução com try-catch-finally

Saída esperada

```
1. Dentro do try
3. Dentro do catch: / by zero
4. Dentro do finally: executa sempre
5. Depois do try-catch-finally
```

Verifique seu entendimento

Se **nenhuma** exceção ocorrer dentro do `try`, o bloco `finally` ainda assim é executado? E o bloco `catch`?

Parte 3: Lançando suas próprias exceções

A palavra-chave `throw`

Definição: `throw`

A palavra-chave **`throw`** **lança** manualmente uma exceção, interrompendo a execução do método naquele exato ponto. É assim que o **próprio programador** sinaliza que algo está errado — não apenas o Java detectando um erro sozinho.

```
1 public class Validador {
2     public static void verificarIdade(int idade) {
3         if (idade < 0) {
4             throw new IllegalArgumentException("Idade nao pode ser
negativa: " + idade);
5         }
6         System.out.println("Idade valida: " + idade);
7     }
8 }
9
10 public class Main {
11     public static void main(String[] args) {
12         Validador.verificarIdade(20); // ok, imprime a mensagem
13         Validador.verificarIdade(-5); // lanca a excecao aqui
14         System.out.println("Esta linha nunca executa");
15     }
16 }
```

Listing 6: Lançando uma exceção manualmente

Saída no terminal

```
Idade valida: 20
Exception in thread "main"
java.lang.IllegalArgumentException: Idade nao pode ser negativa: -5
    at Validador.verificarIdade(Validador.java:4)
    at Main.main(Main.java:11)
```

`throw` sempre acompanha um `new`

`throw` lança um **objeto** de exceção — por isso, sempre vem acompanhado de `new`: `throw new IllegalArgumentException("mensagem")`. A mensagem passada no construtor pode ser recuperada depois com `e.getMessage()`.

Onde colocar o try-catch: quem lança, quem trata

No exemplo anterior, o método `verificarIdade` lança a exceção, mas **não** tem nenhum `try-catch` dentro dele — por isso o programa travou quando chamamos o método com uma idade inválida, sem nenhuma proteção. Isso leva a uma dúvida muito comum de quem está começando: **onde exatamente o try-catch deve ficar?**

Antes de responder, vale entender uma peça importante: a **assinatura** do método.

Definição: Assinatura de um método

A **assinatura** de um método é a sua “porta de entrada”: a primeira linha da declaração, com o modificador de acesso, o tipo de retorno, o nome do método e os parâmetros que ele recebe — tudo o que vem **antes** das chaves `{ }` que abrem o corpo do método.

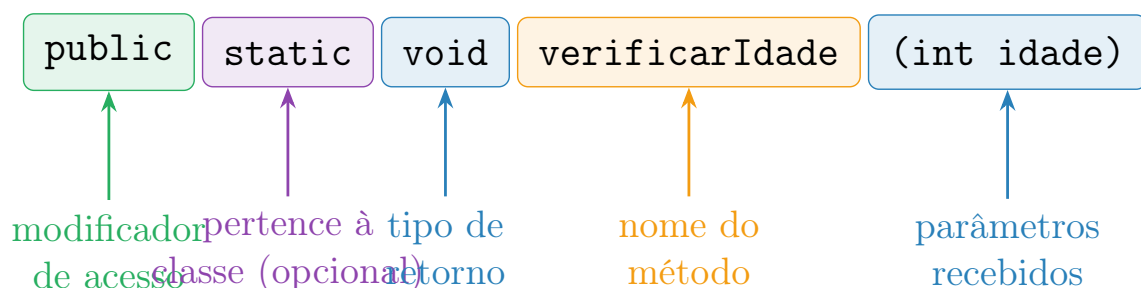


Figure 3: As cinco partes da assinatura de um método: tudo o que aparece antes do `{` que abre o corpo do método.

E o `throws` na assinatura?

Mais adiante, ao estudar exceções *checked*, você verá métodos cuja assinatura inclui também a palavra `throws` (por exemplo, `public void lerArquivo() throws IOException`), avisando quem for chamar o método que ali existe um erro possível. As exceções desta aula (*unchecked*) não exigem essa declaração.

Agora a resposta para a dúvida inicial:

Regra: quem lança não trata o próprio erro

Um método que **detecta** um problema deve apenas **lançar** (`throw`) a exceção e parar por ali — ele não sabe, e não precisa saber, o que fazer quando o erro acontece. Quem decide como reagir é **quem chama** o método, colocando um `try-catch` **ao redor da chamada**, e não dentro do método que lança.

Analogia: o funcionário e o gerente

Um funcionário de caixa que percebe uma nota falsa não decide sozinho o que fazer com o cliente — ele **avisa** o gerente (lança o problema para cima) e é o gerente quem decide como agir. Da mesma forma, o método que detecta o erro apenas avisa (`throw`); quem está “mais acima” na cadeia de chamadas — normalmente quem chamou o método — é quem trata o problema com `try-catch`.

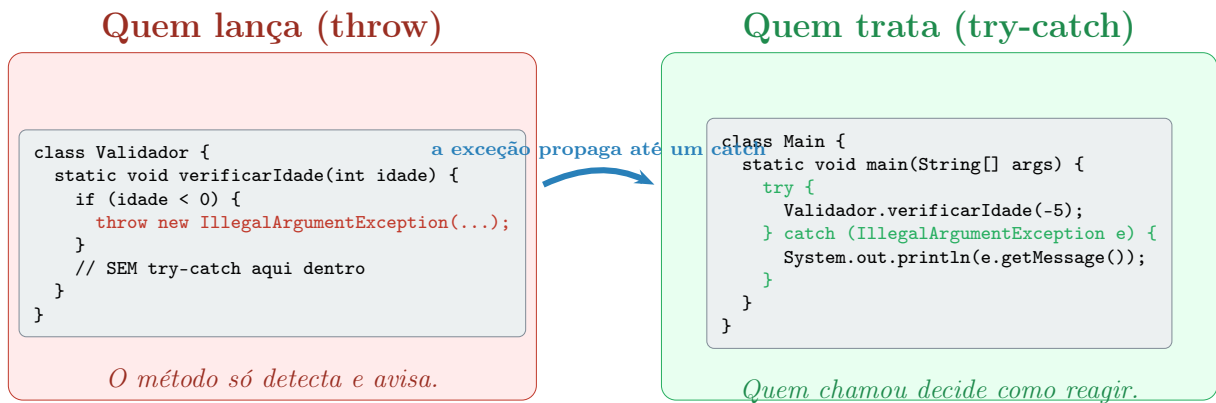


Figure 4: O método que detecta o problema apenas lança a exceção; o try-catch fica em quem chama esse método, não dentro dele.

Onde NÃO colocar o try-catch

Colocar o try-catch dentro do próprio método que lança a exceção, envolvendo o `throw`, quase sempre não faz sentido: o método captura o erro que ele mesmo acabou de criar, e quem chamou nunca fica sabendo que algo deu errado.

```
1 public static void verificarIdade(int idade) {
2     try {
3         if (idade < 0) {
4             throw new IllegalArgumentException("Idade invalida: "
5             + idade);
6         }
7     } catch (IllegalArgumentException e) {
8         System.out.println(e.getMessage()); // o erro morre aqui dentro
9     }
10    // quem chamou este metodo nunca sabe se a idade era invalida!
11 }
```

```
1 public class Validador {
2     // o metodo apenas lanca: nenhum try-catch aqui dentro
3     public static void verificarIdade(int idade) {
4         if (idade < 0) {
5             throw new IllegalArgumentException("Idade nao pode ser
6             negativa: " + idade);
7         }
8         System.out.println("Idade valida: " + idade);
9     }
10 }
11 public class Main {
12     public static void main(String[] args) {
13         // o try-catch fica em volta da CHAMADA, em quem usa o
14         metodo
15     }
16 }
```



```

14         try {
15             Validador.verificarIdade(20);
16             Validador.verificarIdade(-5);
17         } catch (IllegalArgumentException e) {
18             System.out.println("Idade invalida: " + e.getMessage());
19         };
20     }
21     System.out.println("O programa continua normalmente.");
22 }

```

Listing 7: Onde colocar o try-catch: ao redor da chamada, não dentro do método

Saída esperada

```

Idade valida: 20
Idade invalida: Idade nao pode ser negativa: -5
O programa continua normalmente.

```

Pergunta para decidir onde colocar o try-catch

Pergunte: “este método sabe o que fazer se o erro acontecer?”

- Se a resposta for **não** — ele só lança (**throw**) e deixa o problema seguir adiante.
- Quem **sabe** o que fazer (mostrar uma mensagem ao usuário, tentar de novo, cancelar a operação) é quem **chama** o método — e é ali, ao redor da chamada, que entra o **try-catch**.

Verifique seu entendimento

Na classe `Validador`, o método `verificarIdade` tem algum **try-catch** dentro dele?
Por que ele apenas lança (**throw**) em vez de tratar (**try-catch**) o próprio erro?

Validando no construtor: da correção silenciosa à exceção

Na aula sobre encapsulamento, o construtor e o setter reaproveitavam um método **private** que **corrigia** valores inválidos para um padrão seguro (como `validaTitular`, retornando “Não informado” quando o dado era inválido). Isso resolve o problema de duplicar código, mas tem uma desvantagem: **o erro passa despercebido** — o objeto é criado mesmo assim, só que com um dado diferente do que foi pedido.

Agora que conhecemos exceções, podemos melhorar essa estratégia: em vez de **corrigir** o valor, o método de validação pode **lançar uma exceção**, impedindo que o objeto seja criado com dados inválidos.

Antes: correção silenciosa

```
this.titular = validaTitular(titular);  
  
// se for invalido, vira "Nao informado"  
// o objeto E CRIADO mesmo assim  
// ninguem fica sabendo do problema
```

O erro pode passar despercebido.

evoluir

Agora: lança uma exceção

```
validarTitular(titular);  
this.titular = titular;  
  
// se for invalido, uma execucao e lancada  
// o objeto NAO chega a ser criado  
// quem chamou precisa tratar o erro
```

O erro é sinalizado imediatamente.

Figure 5: Duas estratégias diante de um dado inválido: corrigir silenciosamente ou interromper a criação do objeto com uma exceção.

O que muda no método auxiliar privado

A ideia de um método **private** reaproveitado pelo construtor e pelo setter **continua exatamente a mesma**. Só muda o que ele faz com um valor inválido: em vez de **return** de um valor corrigido, ele agora **throws** uma exceção — e por isso não precisa mais devolver nada (**void**).

```
1 public class ContaBancaria {  
2  
3     private String titular;  
4     private int numero;  
5     private double saldo;  
6  
7     public ContaBancaria(String titular, int numero, double  
saldoInicial) {  
8         validarTitular(titular);  
9         validarSaldoInicial(saldoInicial);  
10        this.titular = titular;  
11        this.numero = numero;  
12        this.saldo = saldoInicial;  
13    }  
14  
15    // metodo PRIVADO: lanca execucao se o titular for invalido  
16    private void validarTitular(String titular) {  
17        if (titular == null || titular.isEmpty()) {  
18            throw new IllegalArgumentException("Titular nao pode  
ser vazio");  
19        }  
20    }  
21  
22    // metodo PRIVADO: lanca execucao se o saldo inicial for  
invalido  
23    private void validarSaldoInicial(double saldoInicial) {  
24        if (saldoInicial < 0) {  
25            throw new IllegalArgumentException("Saldo inicial nao
```

```

26     pode ser negativo");
27     }
28
29     public String getTitular() { return titular; }
30     public double getSaldo() { return saldo; }
31
32     // SETTER: reaproveita o MESMO metodo de validacao do
33     construtor
34     public void setTitular(String novoTitular) {
35         validarTitular(novoTitular);
36         this.titular = novoTitular;
37     }
38
39     public void depositar(double valor) {
40         if (valor > 0) saldo += valor;
41     }
42
43     public void mostrarDados() {
44         System.out.println("Titular: " + titular + " | Saldo: R$ "
45 + saldo);
46     }
47 }

```

Listing 8: ContaBancaria validando com exceção em vez de valor padrão

```

1 public class Main {
2     public static void main(String[] args) {
3         try {
4             ContaBancaria conta1 = new ContaBancaria("Maria", 1001,
5             500.0);
6             System.out.println("Conta criada: " + conta1.getTitular
7             ());
8
9             ContaBancaria conta2 = new ContaBancaria("", 1002,
10            100.0); // invalido!
11            System.out.println("Esta linha nunca executa");
12        } catch (IllegalArgumentException e) {
13            System.out.println("Erro ao criar conta: " + e.
14            getMessage());
15        }
16
17        System.out.println("O programa continua.");
18    }
19 }

```

Listing 9: Tratando a criação de contas no main

Saída esperada

```

Conta criada: Maria
Erro ao criar conta: Titular nao pode ser vazio

```

O programa continua.

Verifique seu entendimento

No exemplo acima, o que aconteceria se a chamada a `new ContaBancaria(...)` não estivesse dentro de um bloco `try`?

Checked vs. Unchecked: um panorama rápido

Nem todas as exceções em Java são iguais. Elas se dividem em duas grandes famílias:

	Precisa tratar?	Quando usar	Exemplos
Checked (Exception)	Sim — o compilador exige <code>try-catch</code> ou a declaração <code>throws</code> .	Erros esperados e fora do controle do programa (arquivo não encontrado, falha de rede).	<code>IOException</code>
Unchecked (<code>RuntimeException</code>)	Não — o tratamento é opcional.	Erros de programação, geralmente evitáveis com validação (dado inválido, divisão por zero).	<code>IllegalArgumentException</code> <code>ArithmeticException</code>

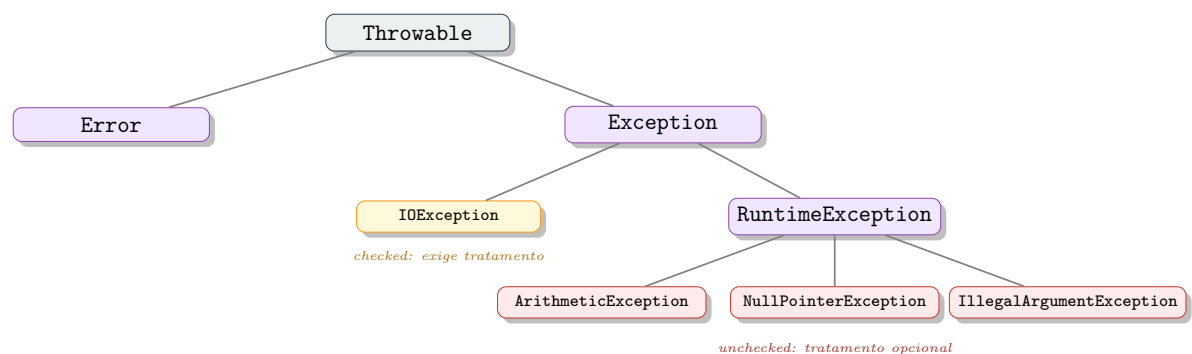


Figure 6: Hierarquia simplificada de exceções em Java: exceções *checked* (como `IOException`) obrigam tratamento; exceções *unchecked* (subclasses de `RuntimeException`) não.

Qual usar por enquanto?

Para erros de **validação de dados** (como os exemplos desta aula), use `IllegalArgumentException` ou uma exceção personalizada *unchecked* (explicada a seguir). Exceções *checked* serão aprofundadas mais adiante, quando o curso tratar de arquivos e outras operações que podem falhar por motivos fora do controle do programa.

Parte 4: Criando classes de exceção personalizadas

Por que criar uma exceção personalizada?

Definição: Exceção personalizada

Uma **exceção personalizada** é uma classe criada pelo próprio programador que **extends** uma exceção já existente (geralmente `RuntimeException`). Ela representa um erro **específico do seu domínio**, com um nome que já explica o que aconteceu.

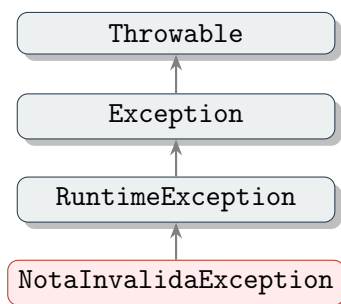
Analogia: etiquetas específicas em vez de uma caixa “diversos”

Imagine uma oficina que joga todas as peças quebradas na mesma caixa “problemas”. Toda vez que algo dá errado, é preciso abrir a caixa inteira e investigar peça por peça para descobrir o que houve. Agora imagine caixas etiquetadas: “motor”, “freio”, “elétrica”. Uma exceção personalizada é essa etiqueta: em vez de lançar sempre um genérico `IllegalArgumentException`, criamos `NotaInvalidaException`, que já diz exatamente qual é o problema.

Construindo e usando a exceção personalizada

```
1 public class NotaInvalidaException extends RuntimeException {  
2  
3     public NotaInvalidaException(String mensagem) {  
4         super(mensagem); // repassa a mensagem para a classe pai  
5     }  
6 }
```

Listing 10: Classe de exceção personalizada



A seta aponta para a superclasse (`extends`): `NotaInvalidaException` herda de `RuntimeException`.

Figure 7: Uma exceção personalizada estende uma classe já existente da hierarquia, herdando seu comportamento.

```
1 public class Aluno {  
2  
3     private String nome;  
4     private int matricula;  
5     private double nota1;  
6     private double nota2;  
7 }
```

```

8     public Aluno(String nome, int matricula, double nota1, double
    nota2) {
9         this.nome = nome;
10        this.matricula = matricula;
11        validarNota(nota1);
12        validarNota(nota2);
13        this.nota1 = nota1;
14        this.nota2 = nota2;
15    }
16
17    // metodo PRIVADO: lanca excecao personalizada se a nota for
    invalida
18    private void validarNota(double nota) {
19        if (nota < 0 || nota > 10) {
20            throw new NotaInvalidaException(
21                "Nota invalida: " + nota + " (deve estar entre 0 e
    10)");
22        }
23    }
24
25    public String getNome() { return nome; }
26    public double getNota1() { return nota1; }
27    public double getNota2() { return nota2; }
28
29    // Setters reaproveitam o MESMO metodo de validacao do
    construtor
30    public void setNota1(double nota1) {
31        validarNota(nota1);
32        this.nota1 = nota1;
33    }
34    public void setNota2(double nota2) {
35        validarNota(nota2);
36        this.nota2 = nota2;
37    }
38
39    public double calcularMedia() {
40        return (nota1 + nota2) / 2;
41    }
42
43    public void mostrarDados() {
44        System.out.printf("%s | media: %.1f%n", nome, calcularMedia
    ());
45    }
46 }

```

Listing 11: Classe Aluno lançando a exceção personalizada

```

1 public class Main {
2     public static void main(String[] args) {
3         try {
4             Aluno a1 = new Aluno("Joao", 101, 8.0, 9.0);
5             a1.mostrarDados();

```

```

6         Aluno a2 = new Aluno("Ana", 102, 8.0, 15.0); // nota
7         invalida!
8         a2.mostrarDados();
9     } catch (NotaInvalidaException e) {
10        System.out.println("Nao foi possivel criar o aluno: " +
11        e.getMessage());
12    }
13    System.out.println("Programa encerrado normalmente.");
14 }
15 }

```

Listing 12: Tratando a exceção personalizada no main

Saída esperada

```

Joao | media: 8.5
Nao foi possivel criar o aluno: Nota invalida: 15.0 (deve estar entre 0 e 10)
Programa encerrado normalmente.

```

Por que capturar `NotaInvalidaException` e não apenas `Exception`?

Capturar o tipo específico `NotaInvalidaException` deixa claro, só de ler o código, que estamos tratando um problema de **nota inválida** — e não qualquer outro erro inesperado que porventura aconteça no bloco `try`.

Boas práticas e erros comuns

Erros comuns ao trabalhar com exceções

1. Bloco catch vazio (“engolir” a exceção).

```
1 try {  
2     conta.sacar(valor);  
3 } catch (Exception e) {  
4     // nada aqui -- PROBLEMA! o erro desaparece  
5     silenciosamente  
6 }
```

Se algo deu errado, ignorar o erro em silêncio só adia (e esconde) o problema.

2. Capturar sempre Exception genérica. Isso mistura erros completamente diferentes no mesmo tratamento e dificulta saber, de fato, o que aconteceu.

3. Usar exceção para controle de fluxo normal. Exceções são para situações excepcionais, não para decisões comuns do programa (por exemplo, não lance uma exceção só para verificar se um array está vazio quando um simples if resolveria).

4. Lançar exceções sem mensagem descritiva. throw new RuntimeException(); não ajuda ninguém a entender o problema. Sempre inclua uma mensagem clara.

Resumo geral dos conceitos

Conceito	Explicação simples	Pergunta-chave
Exceção	Objeto que representa um erro em tempo de execução.	O que deu errado?
try	Bloco de código que pode lançar uma exceção.	O que estou protegendo?
catch	Bloco que trata uma exceção específica.	Como eu reajo ao erro?
finally	Bloco que executa sempre, com ou sem exceção.	O que precisa rodar de qualquer jeito?
throw	Lança manualmente uma exceção.	Onde eu sinalizo o erro?
RuntimeException	Exceção <i>unchecked</i> : tratamento opcional.	Isso é um erro de programação evitável?
Exceção personalizada	Classe própria que estende uma exceção existente.	Esse erro merece um nome específico?

Atividade prática

Escolha **um** dos temas abaixo (podem ser os mesmos das aulas anteriores) e implemente as classes pedidas, aplicando os conceitos desta aula. Sua implementação deve atender a todos os requisitos da lista de verificação abaixo, independentemente do tema escolhido.

Requisitos obrigatórios (valem para qualquer tema)

- Crie **pelo menos uma exceção personalizada**, estendendo `RuntimeException`, com um construtor que recebe e repassa a mensagem via `super(mensagem)`.
- Crie **método(s) privado(s)** no formato `validarXxx(valor)` que lançam a exceção quando o valor for inválido — reaproveitados **tanto no construtor quanto no(s) setter(s)**.
- O construtor **nunca** deve chamar o setter (relembre a regra da aula de encapsulamento).
- No Main de teste, use `try-catch` para criar **pelo menos um objeto válido e um objeto inválido**, mostrando as duas saídas.
- Use `e.getMessage()` para exibir uma mensagem clara sobre o erro capturado.
- Use **arrays** (não coleções), como nos exemplos desta aula.

Tema 1. Sistema de produtos em uma loja

- **Atributos sugeridos:** nome, preco, quantidadeEmEstoque.
- **Exceção sugerida:** `PrecoInvalidoException`.
- **Regra de validação:** preco não pode ser negativo (método privado `validarPreco`, usado no construtor e em `setPreco`).

Tema 2. Sistema de funcionários em uma empresa

- **Atributos sugeridos:** nome, cargo, salario.
- **Exceção sugerida:** `SalarioInvalidoException`.
- **Regra de validação:** salario não pode ser negativo nem zero (método privado `validarSalario`, usado no construtor e em `setSalario`).

Tema 3. Sistema de notas de uma turma

- **Atributos sugeridos:** nome, matricula, nota1, nota2.
- **Exceção sugerida:** `NotaInvalidaException` (a desta aula).
- **Regra de validação:** cada nota deve estar entre 0 e 10 (método privado `validarNota`, reaproveitado no construtor e em cada setter de nota).

Tema 4. Sistema de contas bancárias

- **Atributos sugeridos:** titular, numero, saldo.
- **Exceção sugerida:** `SaldoInvalidoException`.

- **Regra de validação:** o saldo inicial não pode ser negativo (método privado `validarSaldoInicial`, usado no construtor). Além disso, o método `sacar(valor)` deve lançar a exceção quando o valor solicitado for maior que o saldo disponível.

Tema 5. Sistema de jogadores em um time

- **Atributos sugeridos:** `nome`, `posicao`, `golsMarcados`, `jogosDisputados`.
- **Exceção sugerida:** `EstatisticaInvalidaException`.
- **Regra de validação:** `golsMarcados` e `jogosDisputados` não podem ser negativos (método privado `validarQuantidade`, reaproveitado no construtor e nos setters correspondentes).

Antes de programar, planeje no papel

- Qual é o nome da sua exceção personalizada? Que mensagem ela deve carregar para ser realmente útil a quem for lê-la?
- Quais atributos precisam de validação? O método `validarXxx` vai lançar a exceção ou apenas verificar e retornar (sem valor de retorno, já que agora ele lança em vez de corrigir)?
- No seu `Main`, qual situação você vai usar para **provocar** a exceção de propósito e testar o `catch`?
- Depois de capturar a exceção, o programa deve continuar rodando normalmente. O que ele deve fazer em seguida?

Verifique seu entendimento

Depois de implementar sua classe, revise o código: os métodos `validarXxx` são `private`? Eles são chamados tanto no construtor quanto nos setters correspondentes? Existe algum lugar em que um objeto poderia ser criado com dado inválido sem lançar exceção?